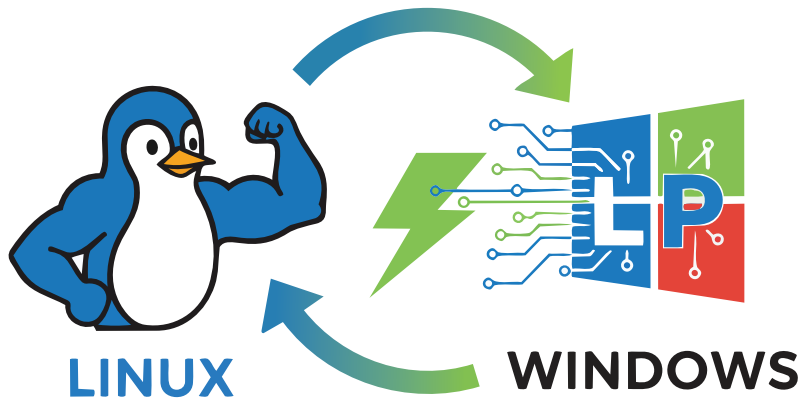


Bringing Linux Power to Windows by Building Custom Commands

This brief tutorial will help you build Python versions of two vital Linux commands—*ls* and *grep*.



As developers, we often find ourselves switching between Linux and Windows environments. While Windows has made strides with PowerShell and WSL (Windows Subsystem for Linux), sometimes you just want the familiar UNIX-like commands available natively. That is why I created Python-based implementations of two essential Linux commands—*ls* and *grep*—and deployed them as native Windows executables.

Why recreate these commands?

Windows has *dir* instead of *ls* and *findstr* instead of *grep*, but their syntax and behaviour differ significantly from their UNIX counterparts. For developers who work across platforms, having consistent commands can significantly improve workflow efficiency. By creating Python versions and converting them to executables with PyInstaller, we get:

- **Cross-platform consistency:** Same syntax across Windows and Linux.
- **Easy customisation:** Python code is simple to modify.
- **Native performance:** Compiled executables run directly without Python interpreter overhead.
- **System-wide availability:** Placed in System32 for global access.

The *ls* command: Windows directory listing, UNIX style

The *ls* command lists directory contents with UNIX-

like formatting and options. Let's break down the key components.

The core functions are:

```
get_file_attributes(filepath)
def get_file_attributes(filepath):
    """Get Windows file attributes"""
```

This function retrieves Windows-specific file attributes (Hidden, System, ReadOnly, Archive) using the *stat* module. It returns a compact string representation like 'HSR' for a hidden, system, read-only file.

```
format_permissions(mode)
python
def format_permissions(mode):
    """Format file permissions for Windows"""
```

The above function converts file mode bits into UNIX-style permission strings (e.g., *-rwx*). While Windows permissions are more complex than UNIX, this provides a familiar visual representation showing directory status and basic read/write/execute flags.

```
format_size(size)
def format_size(size):
    """Format file size in human readable format"""
```